UNIVERSIDAD ARGENTINA DE LA EMPRESA · DESARROLLO DE APLICACIONES II

Planta Libre

Sistema de reserva de espacios de trabajo

Documento técnico · Entrega obligatoria N.º 1 — 1.º parcial · 17 de septiembre de 2026

INTEGRANTES

Berrotarán, Pedro

Checa Orihuela, Keila Nicole

Lussoro, Nicolás

REPOSITORIOS

github.com/pedroberro/Back-Desarrollo-de-Aplicaciones-II

github.com/pedroberro/Front-Desarrollo-de-Aplicaciones-II

APLICACIÓN PUBLICADA

planta-libre-ashen.vercel.app

planta-libre-api.onrender.com

1. Qué es el sistema y qué abarca esta entrega

Planta Libre es un sistema para reservar espacios de trabajo. Una persona crea su cuenta, confirma su email, administra su perfil y sus métodos de pago, y con eso queda habilitada para reservar. El catálogo de espacios y las reservas quedan para la próxima entrega.

Esta entrega cubre la identidad del sistema, que es la base sobre la que se apoya todo lo demás: sin usuarios confirmados y sin control de acceso no tiene sentido reservar nada. Se eligió ese recorte porque permite mostrar componentes terminados de punta a punta, con su interfaz de usuario, su API y su base de datos, en lugar de varios componentes a medias.

Lo entregado son **tres componentes completos y desplegados**: identidad y acceso, usuarios, y medios de pago. Los tres están en funcionamiento en internet, con la aplicación web en Vercel, la API en Render y la base de datos en Neon.

---

2. Arquitectura general

El backend es un monolito modular en capas. Cada pedido HTTP atraviesa siempre el mismo recorrido:

```
Navegador (React)
      ↓ HTTPS /api
Filtros de seguridad  →  valida el token, arma la identidad del usuario
      ↓
Controladores REST    →  reciben y validan el JSON, devuelven códigos HTTP
      ↓
Servicios             →  reglas de negocio y transacciones
      ↓
Repositorios (JPA)    →  consultas y escritura
      ↓
PostgreSQL
```

Las reglas de dependencia son estrictas y se cumplen en el código:

- Los controladores dependen de **interfaces** de servicio, nunca de repositorios ni de entidades.
- Los servicios concentran las reglas y abren las transacciones. Son los únicos que usan repositorios.
- Las entidades JPA no salen nunca de la capa de datos: hacia afuera viajan DTOs.
- Las integraciones externas, hoy el correo, se usan a través de una interfaz propia, no con la clase del proveedor.

Se eligió un monolito modular y no microservicios porque el sistema es chico y lo que se busca son fronteras claras, no despliegues independientes. Las fronteras están expresadas con interfaces y paquetes, así que si en el futuro un componente necesita separarse, el corte ya está marcado. El costo de operar varios servicios hoy no se justificaría.

El frontend es una aplicación React con Vite, TypeScript y Tailwind. Consume la misma API y no tiene lógica de negocio propia: valida los formularios con las mismas reglas que el backend, para avisar antes de enviar, pero el backend vuelve a validar todo.

---

3. Los tres componentes

Cada componente tiene sus propias clases en cada capa. Ninguno accede a los datos de otro por la base: si necesita algo, lo pide por la interfaz del servicio correspondiente.

Componente	Presentación	Negocio	Datos
Identidad y acceso	`AuthController`, DTOs de `dto.auth`, `JwtAuthenticationFilter`, `GlobalExceptionHandler`	`AuthService` con `AuthServiceImpl`, `JwtTokenProvider`, `LimitadorDeIntentos`, `EmailService`	`UsuarioRepository`, `EmailConfirmationTokenRepository`, `PasswordResetTokenRepository`, `TokenBlacklistRepository`
Usuarios	`UsuarioController`, DTOs de `dto.usuario`	`UsuarioService` con `UsuarioServiceImpl`, `UsuarioMapper`	`UsuarioRepository`, entidad `Usuario`
Medios de pago	`MedioPagoController`, DTOs de `dto.medioPago`	`MedioPagoService` con `MedioPagoServiceImpl`	`MedioPagoRepository`, entidad `MedioPago`

Identidad y acceso resuelve el registro, la confirmación del email, el inicio y cierre de sesión, el cambio de contraseña y la recuperación por mail. Es el componente con más reglas: email único, cuentas que nacen pendientes hasta confirmarse, links de un solo uso, links de recuperación que vencen a los 30 minutos y tokens que quedan revocados al cerrar sesión.

Usuarios expone el perfil propio, la edición del teléfono y el listado administrativo de cuentas. El nombre y el email no se modifican, porque el email es la identidad con la que se inicia sesión.

Medios de pago administra las tarjetas del usuario: alta, edición del titular y el vencimiento, elección de la principal y baja. Hay tres reglas propias: hasta tres tarjetas por usuario, no se aceptan tarjetas vencidas, y siempre hay una principal, que se reasigna sola si se borra la que lo era. No se procesan cobros y no se guarda el número completo ni el código de seguridad: solo el titular, el vencimiento y los últimos cuatro dígitos.

---

4. Componente con estado y componentes sin estado

La API no guarda sesión del lado del servidor: Spring Security está configurado como stateless y la identidad viaja en el token en cada pedido. Sobre esa base hay un único componente que sí guarda estado entre pedidos.

Con estado: `LimitadorDeIntentos`. Cuenta cuántas veces cada cliente llamó a las operaciones públicas que pueden abusarse: consultar si un email está registrado, pedir un link de recuperación y reenviar la confirmación. Esa cuenta vive en memoria y depende de los pedidos anteriores del mismo cliente, o sea que dos llamadas idénticas pueden dar resultados distintos según lo que pasó antes. Es el propio contenedor de Spring el que administra su vida: crea una única instancia, llama a su método de inicio anotado con `@PostConstruct`, que levanta la tarea que descarta los contadores vencidos, y al apagar la aplicación llama al método anotado con `@PreDestroy`, que detiene esa tarea y borra el estado. Ambos momentos quedan registrados en el log con el prefijo `[ciclo de vida]`.

Sin estado: los servicios de negocio y el de correo. `AuthServiceImpl`, `UsuarioServiceImpl`, `MedioPagoServiceImpl` y los dos servicios de mail reciben en cada llamada todo lo que necesitan; lo que tienen que recordar está en la base de datos. Sus atributos son dependencias y configuración fijadas al arrancar, no datos de un usuario. Por eso da igual qué instancia atienda cada pedido y pueden atender varios

a la vez. El servicio de correo también es creado e inicializado por el contenedor, que llama a su método anotado con `@PostConstruct` una vez que la configuración está inyectada; ahí el servicio avisa en el log si el envío quedó activo o no.

La diferencia tiene una consecuencia concreta: si mañana se levanta más de una instancia de la API, los servicios sin estado funcionan sin cambios, mientras que el limitador contaría por separado en cada instancia y habría que moverlo a un almacenamiento compartido.

---

5. Patrones de diseño aplicados

Repository (DAO)

Dónde: los cinco repositorios del paquete `domain.repository`. **Por qué:** concentra el acceso a datos y deja los servicios escritos en términos del dominio, sin SQL ni detalles de JPA. Las consultas se declaran por su nombre, como buscar un usuario por email o traer las tarjetas activas de un usuario ordenadas con la principal primero. Si una consulta cambia, se cambia en un solo lugar, y las reglas de negocio se pueden probar con repositorios simulados.

Facade

Dónde: `AuthServiceImpl`. **Por qué:** registrar un usuario implica validar que el email no exista, guardar la cuenta como pendiente, cifrar la contraseña, dar de alta la tarjeta si vino una, generar un token de confirmación y mandar un mail. El controlador no debería conocer ese orden. La fachada lo encapsula detrás de una operación y, de paso, define el límite de la transacción.

Strategy

Dónde: la interfaz `EmailService`, con dos implementaciones: `SmtpEmailService` y `GmailApiEmailService`. **Por qué:** el hosting gratuito donde corre la API bloquea los puertos de SMTP, así que ahí los mails tienen que salir por la API de Gmail sobre HTTPS, mientras que en desarrollo conviene el SMTP de siempre. En lugar de llenar el código de condicionales, cada forma de enviar es una clase y la activa se elige por configuración, con la variable `MAIL_PROVIDER`. El resto del sistema pide "enviá este mail" y no sabe cuál está activa.

Template Method

Dónde: `AbstractEmailService`, clase base de las dos anteriores. **Por qué:** los textos de los mails, la decisión de escribir el link en el log cuando no hay credenciales y el manejo de errores son iguales para las dos formas de enviar; lo único distinto es el envío en sí. La clase base fija esos pasos y deja un único hueco que cada implementación completa. Así, agregar un tercer proveedor no obliga a repetir los textos ni el tratamiento de fallas.

Adapter

Dónde: `CustomUserDetailsService` y `UsuarioMapper`. **Por qué:** Spring Security no sabe nada de la entidad `Usuario`: exige su propia interfaz. El primero traduce el usuario del dominio a lo que el framework espera, incluidos el rol y los permisos, sin que el modelo se contamine con clases del framework. El segundo, generado con MapStruct, traduce la entidad al DTO que ve el cliente, y es lo que evita que el hash de la contraseña salga en una respuesta.

Builder

Dónde: las entidades del dominio, mediante Lombok. **Por qué:** varias se crean con muchos campos, algunos opcionales y otros con valor por defecto, como el estado inicial o la marca de principal. Construirlas paso a

paso y con el nombre de cada campo evita constructores largos donde es fácil equivocar el orden de dos parámetros del mismo tipo.

---

6. Seguridad

Autenticación. Las contraseñas se guardan con BCrypt de costo 12, nunca en claro. Al iniciar sesión, la API devuelve un token JWT firmado que vale 24 horas y viaja en cada pedido. Al cerrar sesión, el token queda registrado como revocado y el filtro lo rechaza desde ese momento, aunque todavía no haya vencido.

Autorización por rol, declarativa. Las reglas están al lado de lo que protegen, no desparramadas en el código. `SecurityConfig` declara qué rutas son públicas: registro, login, confirmación, recuperación y el chequeo de salud; todo lo demás exige un token válido. Sobre eso, la operación sensible `GET /api/usuarios`, que lista todas las cuentas, exige además el rol de administrador con `@PreAuthorize`. Un cliente autenticado recibe 403 y un visitante sin token, 401. Cada rol tiene además una lista de permisos, que la API devuelve junto con el usuario para que la interfaz sepa qué mostrar; los permisos también quedan registrados como autoridades, así una regla futura puede exigir un permiso puntual en vez del rol completo.

Cuidados adicionales.

- Las cuentas nacen pendientes y no pueden iniciar sesión hasta confirmar el email. La verificación se hace después de comprobar la contraseña, para que un intento con contraseña incorrecta no revele que la cuenta existe.
- "Olvidé mi contraseña" responde siempre lo mismo, exista o no la cuenta, y el envío del mail ocurre en segundo plano para que el tiempo de respuesta tampoco delate esa diferencia.
- De los links enviados por mail solo se guarda su hash, igual que con las contraseñas. Quien lea la base no puede usarlos.
- Las operaciones públicas que mandan mails o revelan si un email existe tienen límite de intentos por dirección IP.
- Los errores de validación y de negocio devuelven un mensaje entendible y el código HTTP que corresponde, sin filtrar detalles internos. Las fallas inesperadas se registran en el log y hacia afuera responden un mensaje genérico.
- Los orígenes permitidos para llamar a la API se configuran por variable de entorno, y ningún secreto está escrito en el código.

---

7. Decisiones de diseño y sus motivos

Decisión	Alternativa descartada	Motivo
Sesiones con JWT	Sesión en memoria del servidor	El backend puede reiniciarse o multiplicarse sin perder sesiones, y el frontend vive en otro dominio. El costo es que un token sigue siendo válido hasta que vence, y por eso se agregó la lista de revocados.
Confirmación de email obligatoria	Permitir usar la cuenta sin confirmar	Asegura que la dirección existe y es de quien se registra, que es justamente el dato con el que después se recupera la contraseña.
Link de confirmación de un solo uso y sin vencimiento	Link con vencimiento corto	Se priorizó que nadie quede afuera por abrir el mail tarde. Que sirva una sola vez evita que quede como una llave permanente para entrar sin contraseña.
Link de recuperación con 30 minutos de vida	Sin vencimiento	Cambia la contraseña, que es una operación más sensible: si el mail se filtra más tarde, el link ya no sirve.
Tarjetas guardadas enmascaradas	Guardar el número completo	El sistema no cobra, así que no necesita el número. Guardar menos datos sensibles elimina el riesgo de perderlos.
Migraciones con Flyway y esquema validado, no generado	Dejar que el ORM cree las tablas	El esquema queda versionado y es igual en cada máquina y en producción. El ORM no puede modificarlo por accidente al cambiar una entidad.
Un solo `application.yml` con valores por defecto locales	Un archivo de configuración por ambiente	Cada valor que cambia entre ambientes se lee de una variable de entorno con el valor local por defecto. El que clona el repositorio lo levanta sin configurar nada, y en producción no hay secretos en el artefacto.
Los servicios se usan a través de interfaces	Llamar directo a las implementaciones	Permite tener dos implementaciones del envío de mails y reemplazarlas por configuración, y probar un componente sin levantar sus dependencias.
Validación repetida en el frontend y el backend	Validar solo en el backend	El backend es el que manda, porque la API es pública. La validación del frontend existe para avisar antes de enviar, y las reglas se escribieron una sola vez de cada lado para que no se desalineen.
Errores con un cuerpo uniforme: mensaje, campo y código	Devolver solo el código HTTP	La interfaz puede mostrar el error en el campo que corresponde y reaccionar a casos puntuales, como ofrecer reenviar el link cuando la cuenta está sin confirmar.

8. Datos

El modelo tiene cinco tablas: usuarios, medios de pago, tokens de confirmación, tokens de recuperación y tokens revocados. Un usuario tiene muchas tarjetas y muchos tokens; las tarjetas se dan de baja lógicamente, para no perder el historial.

El esquema se construye con diez migraciones de Flyway que se aplican solas al arrancar. La última crea la cuenta de administrador, ya que el registro público siempre da de alta clientes. Cada cambio de esquema es una migración nueva: ninguna se edita después de haberse aplicado, porque eso rompería las bases donde ya corrió.

9. Despliegue

Los tres componentes están desplegados y accesibles:

Parte	Plataforma	Notas
Aplicación web	Vercel	Sirve la interfaz y reenvía las llamadas a `/api` hacia la API, con lo que el navegador ve un solo origen.
API	Render	Imagen Docker construida en dos etapas: una compila con Maven y la final solo lleva el Java de ejecución y el artefacto. La plataforma verifica que esté viva con un endpoint de salud.
Base de datos	Neon	PostgreSQL administrado, en la misma región que la API.

La configuración sensible se carga como variables de entorno en la plataforma: datos de la base, secreto de firma del token, credenciales del correo, URL del frontend y orígenes permitidos. En desarrollo, esos valores tienen por defecto los de una máquina local y los secretos se leen de un archivo que no se versiona.

Una restricción del hosting cambió un diseño. El plan gratuito de Render bloquea la salida por los puertos de SMTP, así que el envío por SMTP, que funciona en desarrollo, no sirve en producción. En vez de cambiar de hosting o de pagar, se agregó una segunda implementación que manda por la API de Gmail sobre HTTPS, que sí está permitido, y se eligió con una variable. El resto del sistema no se enteró del cambio, que es exactamente para lo que se había puesto la interfaz.

10. Verificación

- **Pruebas automáticas:** cubren el componente con estado, incluidos el límite por ventana de tiempo, la separación por cliente y por acción, y el descarte del estado al detenerlo, y el envío por la API de Gmail simulando las respuestas de Google.
- **Pruebas manuales sobre la aplicación publicada:** registro con confirmación por mail real, inicio de sesión, bloqueo del acceso sin confirmar, reenvío del link, link ya usado, recuperación de contraseña por mail, edición del perfil, alta y baja de tarjetas, rechazo de tarjeta vencida, y la diferencia entre administrador y cliente en la operación restringida.

11. Límites conocidos y próximos pasos

1. **Cambiar la contraseña no cierra las sesiones ya abiertas.** El sistema guarda el momento del cambio, pero el filtro todavía no lo compara contra la fecha del token. Es el primer pendiente por su impacto en seguridad.
2. **El límite de intentos vive en memoria.** Con más de una instancia habría que moverlo a un almacenamiento compartido.
3. **El listado de administración no tiene paginado.** Con pocos usuarios no molesta; conviene resolverlo antes de que crezca.
4. **Falta el dominio de reservas:** catálogo de espacios, disponibilidad y reservas, que es lo previsto para la próxima entrega. Ahí aparece el problema de dos personas reservando el mismo espacio a la misma hora, que se resuelve con una restricción en la base y una transacción, no en la capa de aplicación.